# WAPT FRAMEWORK

## Web Application Penetration Testing Framework

**Complete Developer & Learning Documentation**

Phases 1–6 · Testing Guide · Postman · Plugin System · Architecture

Version 0.1.0 · April 2026

⚠ **For Authorized Testing Only**

# 1. What Is the WAPT Framework?

WAPT (Web Application Penetration Testing) Framework is a **custom-built security testing tool** you have written from scratch across 6 phases. It automates the process of finding security vulnerabilities in web applications — the same job a professional penetration tester does manually, but done by software.

## 1.1 Why Build This?

Commercial tools like Burp Suite Pro cost $449/year. OWASP ZAP is free but complex. Your framework is:

- **Educational** — you understand every line because you wrote it
- **Customizable** — add any module, any payload, any check
- **Portfolio-ready** — a complete security tool demonstrates real skill
- **Extensible** — the plugin system lets anyone add new tests

## 1.2 What Does It Do? (Plain English)

When you point the framework at a website, it:

1. Gathers intelligence — Who owns it? What subdomains exist? What technology stack?
2. Maps the attack surface — What pages exist? What forms? What URL parameters?
3. Probes for vulnerabilities — SQLi, XSS, broken auth, misconfiguration, IDOR
4. Documents everything — Saves every finding with severity, evidence, remediation
5. Generates a professional report — PDF/HTML/JSON/Markdown for the client

## 1.3 The 6 Phases at a Glance

| Phase | Name | What It Does | Key Modules |
|---|---|---|---|
| 1 | Foundation | Core engine, database, CLI, FastAPI server | engine.py, models.py, cli.py, main.py |
| 2 | Recon | Passive intelligence gathering | DNS, WHOIS, Subdomain, TechDetect, Headers |
| 3 | Scanner | Active attack surface mapping | PortScan, DirBrute, Crawler, FormExtractor |
| 4 | Vulns | Exploit vulnerability classes | SQLi, XSS, Auth, IDOR, Misconfig |
| 5 | Reporting | Professional output generation | HTML, PDF, JSON, Markdown reports |
| 6 | Platform | Production-ready platform | Plugins, Stealth, Dashboard UI, Docker |

# 2. Architecture — How Everything Connects

## 2.1 Folder Structure Explained

Every folder has a specific responsibility. Here is what each one does and why it exists:

```
wapt-framework/
├── core/              ← Brain of the framework
│   ├── engine.py      ← Orchestrates all module runs
│   ├── scope.py       ← Prevents testing out-of-scope targets
│   ├── session.py     ← HTTP client wrapper (all web requests go here)
│   ├── config.py      ← Loads settings from .env + config.yaml
│   ├── stealth.py     ← Rate limiting + request evasion
│   └── security.py    ← API auth + audit logging
│
├── modules/           ← All scanning/testing modules
│   ├── recon/         ← Phase 2: Passive intelligence
│   │   ├── dns.py
│   │   ├── subdomain.py
│   │   ├── tech_detect.py
│   │   ├── whois_lookup.py
│   │   └── headers.py
│   ├── scanner/       ← Phase 3: Active mapping
│   │   ├── port_scan.py
│   │   ├── dir_brute.py
│   │   ├── crawler.py
│   │   └── form_extractor.py
│   ├── vulns/         ← Phase 4: Exploit testing
│   │   ├── sqli.py
│   │   ├── xss.py
│   │   ├── auth.py
│   │   ├── idor.py
│   │   └── misconfig.py
│   └── reporter/      ← Phase 5: Report generation
│       ├── collector.py
│       ├── html_report.py
│       ├── exporters.py
│       └── templates/report.html
│
├── api/               ← FastAPI REST API
│   ├── routes/        ← HTTP endpoint handlers
│   │   ├── targets.py ← /api/targets CRUD
│   │   ├── scans.py   ← /api/scans CRUD + findings
│   │   └── reports.py ← /api/reports generate + download
│   └── models/schemas.py  ← Pydantic request/response models
│
├── db/                ← Database layer
│   ├── database.py    ← SQLAlchemy async engine + session
│   └── models.py      ← All database table definitions
│
├── plugins/           ← Custom/community plugins
│   ├── base.py        ← WAPTPlugin base class
│   ├── loader.py      ← Auto-discovery engine
│   └── example_open_redirect.py
│
├── payloads/          ← Attack payload files
│   ├── sqli.yaml
│   ├── xss.yaml
```

```
│   ├── auth_wordlist.yaml
│   └── wordlists/
│
├── ui/                  ← React frontend dashboard
├── reports/output/      ← Generated report files
├── logs/                ← Application logs
├── cli.py               ← Command-line interface entry point
├── main.py              ← FastAPI app definition
├── config.yaml          ← Default configuration
├── .env                 ← Environment variables (secrets)
└── docker-compose.yml ← One-command deployment
```

## 2.2 Data Flow — What Happens When You Run a Scan

Follow a request from CLI command to database record:

6. **CLI receives command:** python cli.py scan https://target.com --type full
7. **ScanEngine created:** engine.py instantiates with target URL, scan ID, module list
8. **Scope checked:** scope.py blocks any blacklisted domains immediately
9. **HTTP session opened:** session.py creates an httpx.AsyncClient for all requests
10. **Modules execute in order:** each module's run() called, returns a list of finding dicts
11. **Findings saved to DB:** finding_saver.py writes each finding to SQLite after each module
12. **Report generated:** collector.py reads findings from DB, template renders HTML/PDF
13. **CLI prints table:** rich table shows all findings sorted by severity

## 2.3 Database Tables

Three tables store all scan data. Understanding them is key to understanding the whole system:

### Target table — what you are testing

| Column | Type | Purpose |
|---|---|---|
| id | UUID string | Unique identifier for this target |
| name | String | Human-readable name you give it |
| url | String | Base URL of the application |
| description | Text | Optional notes about the app |
| scope_notes | Text | Written permission evidence (critical legally) |
| is_active | Boolean | Soft delete flag |
| created_at | DateTime | When added |

### Scan table — one row per test run

| Column | Type | Purpose |
|---|---|---|
| id | UUID string | Unique scan identifier |

| Column | Type | Purpose |
|---|---|---|
| target_id | FK → Target | Which target this scan tested |
| scan_type | Enum | full / recon / scan / vuln / auth |
| status | Enum | pending / running / completed / failed / cancelled |
| modules_run | JSON | List of module names that executed |
| config_used | JSON | Snapshot of rate_limit, timeout settings |
| started_at | DateTime | When scan execution began |
| finished_at | DateTime | When scan completed |
| error_msg | Text | Error details if scan failed |

### Finding table — one row per vulnerability found

| Column | Type | Purpose |
|---|---|---|
| id | UUID string | Unique finding identifier |
| scan_id | FK → Scan | Which scan discovered this |
| title | String | Short description e.g. "SQL Injection — id parameter" |
| severity | Enum | critical / high / medium / low / info |
| cvss_score | Float | CVSS v3 score 0.0–10.0 |
| vuln_type | String | Machine tag e.g. "sqli", "xss", "missing_security_header" |
| url | String | The URL where the vulnerability was found |
| parameter | String | Which input field/parameter is vulnerable |
| payload_used | Text | Exact payload that triggered the vulnerability |
| evidence | Text | Response snippet proving the vulnerability exists |
| description | Text | Full explanation of the vulnerability |
| remediation | Text | How to fix it |
| references | JSON | List of OWASP/CVE/CWE links |
| confirmed | Boolean | Was the finding automatically verified? |
| is_false_positive | Boolean | Manually marked as false positive? |

# 3. Running the API Server

## 3.1 Your Command Explained

You ran:

```
uvicorn api.main:app --reload --port 8000
```

Here is what each part means:

| Part | What It Means |
|---|---|
| uvicorn | The ASGI server that serves your FastAPI app (like Apache/Nginx for Python) |
| api.main | The Python module path: the "api" folder, "main.py" file |
| :app | The FastAPI instance variable named "app" inside main.py |
| --reload | Auto-restart when you save any .py file (development only, never in production) |
| --port 8000 | Listen on TCP port 8000 |

> ⚠ Issue: The command should be `uvicorn main:app` not `uvicorn api.main:app` because main.py is in the project root, not inside an "api" package.
> Correct command: `uvicorn main:app --reload --port 8000`
> OR from the project root: `python cli.py server`

## 3.2 All URLs Once the Server Is Running

| URL | What It Is | Use It For |
|---|---|---|
| http://localhost:8000 | Root (no route defined) | N/A |
| http://localhost:8000/api/health | Health check endpoint | Verify server is alive |
| http://localhost:8000/api/docs | Swagger UI (auto-generated) | Interactive API testing in browser |
| http://localhost:8000/api/redoc | ReDoc UI (cleaner docs) | Reading API documentation |
| http://localhost:8000/api/targets | Targets REST endpoint | Manage test targets |
| http://localhost:8000/api/scans | Scans REST endpoint | Launch and view scans |
| http://localhost:8000/api/reports | Reports endpoint | Generate and download reports |
| http://localhost:5173 | React dashboard (run separately) | Full web UI |

> ✓ Open http://localhost:8000/api/docs in your browser RIGHT NOW.
> Swagger UI is a built-in interactive testing tool — no Postman needed for basic testing.
> Every API endpoint is listed there with a "Try it out" button.

# 4. Testing from the Browser — Swagger UI

## 4.1 What Is Swagger UI?

FastAPI automatically generates **Swagger UI** at `http://localhost:8000/api/docs`. It reads your Pydantic schemas and creates a fully interactive API explorer. You can send real HTTP requests to your own server directly from the browser — no Postman, no code needed.

## 4.2 Step-by-Step: Test Every Endpoint in Swagger

### Step 1 — Check the Server is Alive

14. Open http://localhost:8000/api/docs in your browser
15. Find the "Health" section at the top
16. Click GET /api/health
17. Click "Try it out" → "Execute"
18. You should see HTTP 200 with: {"status": "ok", "app": "WAPT Framework", "version": "0.1.0"}

### Step 2 — Create Your First Target

19. Find the "Targets" section in Swagger
20. Click POST /api/targets → "Try it out"
21. Replace the request body with:

```
{
  "name": "Test App",
  "url": "http://testphp.vulnweb.com",
  "description": "Acunetix intentionally vulnerable app",
  "scope_notes": "Public test target, permission granted by Acunetix"
}
```

22. Click Execute
23. You get HTTP 201 with the new target including its generated UUID id
24. Copy the id value — you need it for the scan step

### Step 3 — List All Targets

25. Click GET /api/targets → "Try it out" → Execute
26. You will see an array with your target

### Step 4 — Create a Scan

27. Click POST /api/scans → "Try it out"
28. Use this body (paste your target id):

```
{
  "target_id": "paste-your-target-uuid-here",
  "scan_type": "recon"
```

```
}
```

29. Execute — you get a scan record with status "pending"
30. Note: this only REGISTERS the scan in the DB. The scan does not run via the API yet
31. To actually RUN a scan, use the CLI: python cli.py scan http://testphp.vulnweb.com --type recon

> **i** Why does the API only create a record but not run the scan?
> Running a scan is a long operation (30 seconds to many minutes).
> HTTP requests time out after ~30s. The solution is background tasks or WebSockets — this is a Phase 6 enhancement that connects the API scan creation to actually running the engine.

### Step 5 — View Scan Findings

32. Click GET /api/scans/{scan_id}/findings → "Try it out"
33. Enter the scan_id from a completed CLI scan
34. You see all findings from that scan, sorted by severity

### Step 6 — Generate a Report

35. Click POST /api/reports/generate → "Try it out"
36. Use this body:

```
{
  "scan_id": "your-scan-uuid",
  "formats": ["html", "pdf", "json", "markdown"]
}
```

37. You get HTTP 202 Accepted — report generates in the background
38. Wait 5–10 seconds, then call GET /api/reports/list to see the files
39. Use GET /api/reports/download/{filename} to download

# 5. Testing with Postman — Complete Guide

## 5.1 What Is Postman?

Postman is a desktop application for sending HTTP requests to APIs. It is the industry standard tool for **API development and testing**. While Swagger UI works for quick tests, Postman is better for: saving request collections, running automated test sequences, setting environment variables, and sharing API tests with a team.

## 5.2 Setting Up Postman

40. Download Postman from https://www.postman.com/downloads/
41. Install and open it (free account not required for basic use)
42. Click "New Collection" → name it "WAPT Framework"
43. Click the gear icon → "Environments" → "New Environment" → name it "WAPT Local"
44. Add these variables to the environment:

| Variable | Initial Value | Purpose |
|---|---|---|
| base_url | http://localhost:8000/api | Base URL for all requests |
| target_id | (empty) | Filled after creating a target |
| scan_id | (empty) | Filled after creating a scan |

45. Select "WAPT Local" from the environment dropdown (top right)

## 5.3 Complete Postman Request Collection

### Request 1: Health Check

| Field | Value |
|---|---|
| Method | GET |
| URL | {{base_url}}/health |
| Headers | None required |
| Body | None |
| Expected Response | 200 OK — {"status": "ok"} |

### Request 2: Create Target

| Field | Value |
|---|---|
| Method | POST |
| URL | {{base_url}}/targets/ |

| Field | Value |
|---|---|
| Headers | Content-Type: application/json |
| Body (raw JSON) | {"name":"DVWA","url":"http://testphp.vulnweb.com","scope_notes":"Public test target"} |
| Expected Response | 201 Created — target object with id field |
| Post-request Script | pm.environment.set("target_id", pm.response.json().id); |

**Post-request script:** In the "Tests" tab, paste this to auto-save the ID:

```
// Postman "Tests" tab — runs after every request
if (pm.response.code === 201) {
    const body = pm.response.json();
    pm.environment.set("target_id", body.id);
    console.log("Target ID saved:", body.id);
}
```

### Request 3: List All Targets

| Field | Value |
|---|---|
| Method | GET |
| URL | {{base_url}}/targets/ |
| Expected Response | 200 OK — array of target objects |

### Request 4: Get Single Target

| Field | Value |
|---|---|
| Method | GET |
| URL | {{base_url}}/targets/{{target_id}} |
| Expected Response | 200 OK — single target object |

### Request 5: Create Scan Record

| Field | Value |
|---|---|
| Method | POST |
| URL | {{base_url}}/scans/ |
| Body | {"target_id":"{{target_id}}","scan_type":"full"} |
| Expected Response | 201 Created — scan object with status "pending" |
| Save scan_id | pm.environment.set("scan_id", pm.response.json().id); |

### Request 6: Get Scan Status

| Field | Value |
|---|---|
| Method | GET |
| URL | {{base_url}}/scans/{{scan_id}} |
| Expected Response | 200 OK — scan object with current status |

### Request 7: Get Scan Findings

| Field | Value |
|---|---|
| Method | GET |
| URL | {{base_url}}/scans/{{scan_id}}/findings |
| Expected Response | 200 OK — {"findings": [...], "total": N} |

### Request 8: Generate Reports

| Field | Value |
|---|---|
| Method | POST |
| URL | {{base_url}}/reports/generate |
| Body | {"scan_id":"{{scan_id}}","formats":["html","pdf","json","markdown"]} |
| Expected Response | 202 Accepted — {"message":"Report generation started"} |

### Request 9: List Generated Reports

| Field | Value |
|---|---|
| Method | GET |
| URL | {{base_url}}/reports/list |
| Expected Response | 200 OK — {"reports":[...files...],"count":N} |

### Request 10: Download a Report

| Field | Value |
|---|---|
| Method | GET |

| Field | Value |
|---|---|
| URL | {{base_url}}/reports/download/WAPT-XXXXXXXX-20260409.html |
| Expected Response | 200 OK — HTML file content (or PDF binary) |
| Postman tip | Click "Send and Download" to save the file directly |

### Request 11: Report Summary (no file generated)

| Field | Value |
|---|---|
| Method | GET |
| URL | {{base_url}}/reports/{{scan_id}}/summary |
| Expected Response | JSON with overall_risk, stats counts, OWASP coverage |

### Request 12: Delete Target (soft delete)

| Field | Value |
|---|---|
| Method | DELETE |
| URL | {{base_url}}/targets/{{target_id}} |
| Expected Response | 204 No Content |

## 5.4 Running a Complete Test Sequence in Postman

Use Postman's Collection Runner to run all requests in order automatically:

46. In your WAPT collection, click the three dots → "Run collection"
47. Set "Iterations" to 1, delay to 500ms
48. Select all requests in order (Health → Create Target → Create Scan → etc.)
49. Click "Run WAPT Framework"
50. All requests execute in sequence, environment variables pass between them automatically

# 6. Testing from the React Frontend Dashboard

## 6.1 Starting the Frontend

```
# Terminal 1 — Backend (already running)
uvicorn main:app --reload --port 8000

# Terminal 2 — Frontend
cd ui
npm install        # first time only
npm run dev

# Open in browser:
http://localhost:5173
```

## 6.2 Dashboard Pages Explained

### Dashboard (/) — Overview

Shows: total targets count, total scans, running scans, generated reports. Has a severity donut chart and recent scans list. All data fetches automatically on page load.

### Targets (/targets) — Manage Test Targets

What you can do here:

- Click "Add Target" → fill name, URL, description, scope notes → "Create Target"
- See all your targets in a list with their URLs
- Click the trash icon to soft-delete a target
- The scope notes field is important — always document your authorization here

### Scans (/scans) — Launch and View Scans

What you can do here:

- Click "New Scan" → select target → select scan type → select stealth mode → "Launch Scan"
- The scan record is created in the database
- The scan does not start automatically from the UI yet — run it from CLI
- See all scans with their status (pending/running/completed/failed)
- Click any scan row to go to the detail page

### Scan Detail (/scans/:id) — View Results

This is the most important page. It shows:

- Scan status — auto-refreshes every 3 seconds while running
- Severity breakdown — Critical/High/Medium/Low/Info counts
- OWASP Top 10 coverage — which vulnerability categories were found

- "Generate Report" button — triggers all 4 report formats simultaneously

### Reports (/reports) — Download Reports

Lists every generated report file with format badge and file size. Click "Download" to save any report. Auto-refreshes every 10 seconds.

### Plugins (/plugins) — Plugin Registry

Shows all discovered plugins. Currently shows the built-in example (open_redirect). Any plugin you drop into plugins/ appears here after a server restart.

## 6.3 Complete Frontend Workflow

51. Open http://localhost:5173
52. Go to Targets → Add Target → fill in testphp.vulnweb.com → Create
53. Go to Scans → New Scan → select your target → type: recon → Launch Scan
54. Open your terminal: python cli.py scan http://testphp.vulnweb.com --type recon
55. Go back to Scan Detail — watch status update to "completed" automatically
56. Click "Generate Report" → go to Reports tab → download your PDF

# 7. What Is a Plugin? — Complete Explanation

## 7.1 The Concept (Plain English)

A **plugin** is an independent module that adds new scanning capability to the framework **without modifying any existing code**. You write a Python file, drop it in the plugins/ directory, and the framework automatically discovers and runs it alongside built-in modules.

Think of it like browser extensions. Chrome works fine without extensions. But you can install an AdBlocker and it adds capability. You never touched Chrome's source code — it just integrates through a defined interface. Your plugin system works the same way.

## 7.2 Why a Plugin System?

| Problem Without Plugins | Solution With Plugins |
|---|---|
| To add a new check, you edit core modules | Drop a .py file in plugins/ — core stays unchanged |
| Different testers need different checks | Each tester writes their own plugin |
| Framework updates might break custom code | Plugins are isolated — they only use the public API |
| Hard to share checks with the community | Share a single .py file, others drop it in plugins/ |
| All code in one giant file gets messy | Each check is self-contained in its own file |

## 7.3 How Plugin Discovery Works (The Loader)

When a scan starts, the plugin loader runs this process automatically:

57. **Walk plugins/:** find every .py file that is not __init__.py, base.py, or loader.py
58. **Import each file:** Python dynamically imports the module with importlib.import_module()
59. **Inspect for classes:** find every class that inherits from WAPTPlugin
60. **Validate the contract:** check it has a unique name, description, and async run() method
61. **Register it:** add to the plugin registry dict {name: PluginClass}
62. **Run it:** when a scan executes, plugin run() is called just like any built-in module

## 7.4 The Plugin Contract — What Every Plugin Must Have

Every plugin must inherit WAPTPlugin and define these:

```
from plugins.base import WAPTPlugin

class MyPlugin(WAPTPlugin):
```

```
    # REQUIRED — must be unique across all plugins
    name        = "my_plugin_name"

    # REQUIRED — what your plugin does (one sentence)
    description = "Detects open redirects in URL parameters"

    # OPTIONAL but good practice
    version     = "1.0.0"
    author      = "Your Name"
    category    = "vuln"   # recon | scanner | vuln | custom
    tests_for   = ["open_redirect"]   # vulnerability types

    # REQUIRED — must be async, must return list of finding dicts
    async def run(self) -> list:
        findings = []
        # ... your scanning logic ...
        return findings
```

## 7.5 What Plugins Can Access

Because WAPTPlugin inherits from BaseModule, every plugin automatically gets:

| Attribute | Type | What It Is |
|---|---|---|
| self.engine | ScanEngine | The main scan orchestrator — access target URL, scan ID |
| self.engine.target_url | str | The URL being tested: "https://target.com" |
| self.engine.scope | ScopeManager | Check if a URL is in scope before testing it |
| self.engine.session | ScanSession | Pre-configured HTTP client for making requests |
| self.engine.input_registry | dict | All forms and URL params found by the crawler |
| self.config | Settings | All configuration (rate limits, timeouts, user agents) |
| self.make_finding() | method | Helper to build a correctly structured finding dict |

## 7.6 Writing Your Own Plugin — Step by Step

### Example: Write a plugin that detects clickjacking

Clickjacking happens when a site can be embedded in an iframe on another domain. We check the X-Frame-Options header.

```
# File: plugins/clickjacking.py
# Drop this file in plugins/ and it auto-loads on next scan

import httpx
from typing import List
```

```
from plugins.base import WAPTPlugin


class ClickjackingPlugin(WAPTPlugin):
    name        = "clickjacking"
    version     = "1.0.0"
    author      = "Your Name"
    description = "Detects missing clickjacking protection headers"
    category    = "vuln"
    tests_for   = ["clickjacking"]

    async def run(self) -> List[dict]:
        findings = []
        target = self.engine.target_url

        try:
            async with httpx.AsyncClient(
                timeout=self.engine.timeout,
                verify=False,
            ) as client:
                resp = await client.get(target)

            headers = {k.lower(): v for k, v in resp.headers.items()}

            has_xfo = "x-frame-options" in headers
            csp_val = headers.get("content-security-policy", "")
            has_csp_fa = "frame-ancestors" in csp_val.lower()

            if not has_xfo and not has_csp_fa:
                # Use the helper method from WAPTPlugin
                findings.append(self.make_finding(
                    title="Clickjacking Protection Missing",
                    severity="medium",
                    vuln_type="clickjacking",
                    url=target,
                    description=(
                        "Neither X-Frame-Options nor CSP frame-ancestors is
set. "
                        "The page can be embedded in an iframe on any domain. "
                        "Attackers overlay a transparent iframe to trick users
"
                        "into clicking hidden buttons (clickjacking)."
                    ),
                    evidence=f"X-Frame-Options: not present\n"
                             f"CSP frame-ancestors: not present",
                    remediation=(
                        "Add: X-Frame-Options: DENY\n"
                        "Or add to CSP: frame-ancestors none"
                    ),
                    cvss_score=4.3,
                    references=[
                        "https://owasp.org/www-community/attacks/Clickjacking",
                        "https://cwe.mitre.org/data/definitions/1021.html",
                    ],
                ))

        except Exception as e:
            pass  # Never crash — just return empty findings

        return findings
```

**To use it:** save as plugins/clickjacking.py → run any scan → it auto-executes → findings appear in the table

## 7.7 Plugin Best Practices

- **Never crash:** wrap everything in try/except. A crashing plugin aborts the whole scan.
- **Never test out-of-scope URLs:** call self.engine.scope.is_url_in_scope(url) before any request to a discovered URL.
- **Use make_finding():** do not build finding dicts manually — the helper ensures every required field is present.
- **Be specific:** one plugin = one vulnerability class. Do not cram 5 checks into one plugin.
- **Test on safe targets:** use DVWA or testphp.vulnweb.com before testing anything else.
- **Document your plugin:** the description field shows in the UI — make it informative.

# 8. CLI Reference — Every Command Explained

## 8.1 All Available Commands

```
python cli.py --help              # Show all commands
python cli.py info                # Show config and settings
python cli.py server              # Start the FastAPI server
python cli.py scan <url>          # Run a penetration test
python cli.py report <scan_id>    # Generate reports
```

## 8.2 The scan Command — All Options

```
python cli.py scan <TARGET_URL> [OPTIONS]

Options:
  --type      Scan type: full | recon | scan | vuln | auth
              Default: full

  --rate-limit  Requests per second (integer)
              Default: 5 (from config.yaml)

  --timeout   Seconds before a request times out (integer)
              Default: 10 (from config.yaml)

  --stealth   Request evasion mode: normal | polite | stealth
              Default: normal
```

## 8.3 Scan Types Explained

| Type | Modules Run | Use When | Typical Duration |
|---|---|---|---|
| recon | DNS, WHOIS, Subdomain, TechDetect, Headers | First pass — gather intel before active testing | 20–60 seconds |
| scan | PortScan, DirBrute, Crawler, FormExtractor | Map attack surface after recon | 1–5 minutes |
| vuln | SQLi, XSS, Auth, IDOR, Misconfig | Run only vuln tests (needs prior scan for input registry) | 2–10 minutes |
| auth | AuthTester only | Focus on login/auth issues | 30 seconds–2 min |
| full | All modules from all phases | Complete assessment | 5–20 minutes |

## 8.4 Stealth Modes Explained

| Mode | Delay | Concurrency | UA Rotation | Use When |
|---|---|---|---|---|
| normal | 0–0.2s | 20 concurrent | No | Internal testing, CTFs, |

| Mode | Delay | Concurrency | UA Rotation | Use When |
|---|---|---|---|---|
| | | | | known-safe targets |
| polite | 0.5–2.0s | 5 concurrent | Yes | Testing with client — be courteous to the server |
| stealth | 2–8s + jitter | 2 concurrent | Yes + headers | WAF evasion, IDS-sensitive environments |

## 8.5 The report Command — All Options

```
python cli.py report <SCAN_ID> [OPTIONS]

Options:
  --format    html | pdf | json | markdown | all
              Default: all

  --output-dir  Directory to save reports
              Default: reports/output/

Examples:
  python cli.py report abc123 --format pdf
  python cli.py report abc123 --format all --output-dir ~/Desktop
```

## 8.6 Practical Examples

```
# Quick recon scan
python cli.py scan http://testphp.vulnweb.com --type recon

# Full scan with polite rate limiting
python cli.py scan http://testphp.vulnweb.com --type full --stealth polite

# Very slow stealth scan (WAF bypass)
python cli.py scan https://target.com --type full --stealth stealth --rate-
limit 1

# Vuln-only (run after a scan to get input_registry)
python cli.py scan http://testphp.vulnweb.com --type vuln

# Generate PDF report only
python cli.py report 8a6d0172-847c-403c-a515-2555efd2b4a2 --format pdf

# Check your config
python cli.py info
```

# 9. Every Module Explained — What It Does and How

## Phase 2: Reconnaissance Modules

### 9.1 DNS Module (modules/recon/dns.py)

**Purpose:** Resolve all DNS record types for the target domain. Attempt zone transfer.

| Record Type | What It Reveals |
|---|---|
| A | IPv4 address of the server — confirms the host is live |
| AAAA | IPv6 address |
| MX | Mail servers — useful for email spoofing tests |
| NS | Nameservers — used for zone transfer attempt |
| TXT | SPF/DKIM/DMARC records, verification tokens, internal notes |
| CNAME | Aliases — may reveal cloud provider (s3.amazonaws.com etc.) |
| SOA | Zone authority — admin email often exposed |

> What is a Zone Transfer (AXFR)?
> A zone transfer is a DNS feature meant for secondary nameservers to clone zone data from primary.
> If misconfigured, it leaks the ENTIRE DNS database — every subdomain, every IP, all records.
> This is like getting the full map of an organization's infrastructure in one request.
> The module attempts AXFR against each discovered nameserver.

### 9.2 Subdomain Module (modules/recon/subdomain.py)

**Purpose:** Find all subdomains of the target domain using two techniques.

- **Technique 1 — Certificate Transparency Logs:** Every SSL certificate ever issued is publicly logged. We query crt.sh to get them passively (no packets sent to the target).
- **Technique 2 — DNS Wordlist Bruteforce:** Try admin.target.com, api.target.com, etc. by checking if each resolves in DNS. 85 common subdomain prefixes tested concurrently.
- **Subdomain Takeover Check:** If a subdomain points to AWS S3/GitHub Pages/Heroku and returns "not found", an attacker can claim that resource. Flagged as High severity.

### 9.3 Technology Detector (modules/recon/tech_detect.py)

**Purpose:** Identify what software powers the target by checking HTTP headers, cookie names, and HTML body patterns against a signature database.

> Why technology detection matters:
> If we detect WordPress 5.x, we know to look for WordPress-specific vulnerabilities.
> If we detect PHP, we know to try PHP-specific payloads.
> If we detect a WAF (Imperva, Cloudflare), we know to use evasion techniques.
> The tech stack shapes which vulnerability tests are most likely to succeed.

### 9.4 WHOIS Module (modules/recon/whois_lookup.py)

**Purpose:** Look up domain registration data. Key findings: expiring domains (can be sniped), exposed registrant emails (phishing target), privacy protection status.

### 9.5 Header Analyzer (modules/recon/headers.py)

**Purpose:** Audit HTTP response headers for security misconfigurations. Checks 7 security headers plus cookie flags.

| Header | Risk if Missing | Severity |
|---|---|---|
| Strict-Transport-Security (HSTS) | SSL stripping attacks — user on HTTP | High |
| Content-Security-Policy (CSP) | XSS has no browser-level defense | High |
| X-Frame-Options | Clickjacking attacks | Medium |
| X-Content-Type-Options | MIME sniffing / content injection | Low |
| Referrer-Policy | URL leakage to third parties | Low |
| Permissions-Policy | Unauthorized camera/mic/geo access | Low |
| Cross-Origin-Opener-Policy | Cross-window attacks | Info |

## Phase 3: Active Scanner Modules

### 9.6 Port Scanner (modules/scanner/port_scan.py)

**Purpose:** Discover which TCP ports are open and what services run on them. Uses Nmap if installed, falls back to async socket scanning.

> ⚠ Port scanning is loud. It generates network traffic that firewalls, IDS, and SOC teams notice. In a real engagement, confirm with the client that port scanning is in scope. Use --stealth polite or --stealth stealth if you need to be quieter.

### 9.7 Directory Bruteforcer (modules/scanner/dir_brute.py)

**Purpose:** Probe hundreds of common file and directory paths to find hidden content not linked from the main site.

**Smart filtering:** First fetches a known-bad URL to fingerprint the 404 page. Then compares every response body hash against this fingerprint to skip "soft 404s" — pages that return HTTP 200 but are actually error pages.

**High-value targets it finds:** .env files (API keys), .git/ directories (full source code), phpinfo.php (server internals), admin panels, database admin tools, backup archives.

### 9.8 Web Crawler (modules/scanner/crawler.py)

**Purpose:** Systematically visit every page on the site, following links up to 5 levels deep and up to 200 pages. Extracts forms, URL parameters, JavaScript files, email addresses, and HTML comments.

**BFS (Breadth-First Search):** Starts at the target URL, processes all links on that page first, then follows each to the next level. This ensures shallow pages are fully covered before going deep.

### 9.9 Form Extractor (modules/scanner/form_extractor.py)

**Purpose:** Post-processes crawler results into a clean "input registry" — a structured list of every testable input. This registry is what Phase 4 vulnerability modules consume. Think of it as the menu of things to attack.

## Phase 4: Vulnerability Modules

### 9.10 SQL Injection (modules/vulns/sqli.py)

**What it is:** SQL injection is when user input gets inserted directly into a database query, allowing an attacker to read, modify, or delete any data in the database.

| Technique | How It Works | Detection Method |
|---|---|---|
| Error-based | Inject a quote → database crashes and prints its error message | Scan response for known DB error strings |
| Boolean-blind | Inject AND 1=1 (true) vs AND 1=2 (false) → compare page sizes | Significant length difference between true/false responses |
| Time-based | Inject SLEEP(5) → measure if response is delayed 5+ seconds | Response time >= 4 seconds after injection |

### 9.11 XSS — Cross-Site Scripting (modules/vulns/xss.py)

**What it is:** XSS is when user input gets reflected in an HTML page without encoding, allowing an attacker to inject JavaScript that runs in other users' browsers.

**Canary approach:** Instead of directly injecting <script> tags (which may be blocked by WAFs), we first inject a harmless unique string. If it appears unescaped in the response, we know the parameter reflects. Then we escalate to real payloads.

**Context detection:** We detect WHERE in the HTML the value lands (inside a tag body, in an attribute, in a JS string) and choose the most effective payload for that context.

### 9.12 Authentication Tester (modules/vulns/auth.py)

**What it tests:**

- **Default credentials:** Tries 20 username/password combinations like admin/admin

- **Account lockout:** Checks if the app locks accounts after failed attempts
- **Username enumeration:** "User not found" vs "Wrong password" reveals valid usernames
- **JWT analysis:** Decodes any JWT token found, checks for "none" algorithm (sign forgery), weak algorithm (HS256 with crackable secret), and sensitive data in payload

## 9.13 IDOR — Insecure Direct Object Reference (modules/vulns/idor.py)

**What it is:** When you change a number in a URL (/profile/42 → /profile/43) and get someone else's data, that's IDOR. The server trusts the ID without checking if the logged-in user owns that object.

**How we test:** Find URLs with numeric IDs, UUIDs, or MongoDB ObjectIDs. Fetch the original. Change the ID by ±1. If the changed URL also returns HTTP 200 with similar content length, IDOR is likely.

## 9.14 Security Misconfiguration (modules/vulns/misconfig.py)

**What it tests:**

- **HTTP TRACE:** If enabled, enables Cross-Site Tracing to steal cookies despite HttpOnly
- **PUT/DELETE methods:** May allow file upload or deletion if misconfigured
- **Verbose errors:** Stack traces expose file paths, framework versions, internal logic
- **Debug endpoints:** Spring Actuator, Django debug toolbar, phpinfo.php expose internal state
- **Directory listing:** /uploads/ showing file list leaks data
- **Default pages:** "Welcome to nginx", "It Works!" confirm the stack and incomplete setup

# 10. Report Generation — Complete Guide

## 10.1 The Four Report Formats

| Format | Best For | How to Open | File Size |
|---|---|---|---|
| HTML | Sharing with clients, browser viewing | Any browser — double click | ~200KB |
| PDF | Professional delivery, printing | Adobe Reader, browser | ~300KB |
| JSON | Automation, SIEM integration, Jira API | Any code/text editor | ~50KB |
| Markdown | GitHub issues, Confluence, Notion | GitHub, VS Code, Obsidian | ~30KB |

## 10.2 What Each Section of the Report Contains

### Cover Page

Report ID, target name and URL, scan date, scan type, overall risk rating, confidentiality notice.

### Executive Summary

Written in plain English for non-technical stakeholders. States the overall risk, critical finding count, and CVSS score statistics.

### Scan Information Table

Technical metadata: scan ID, duration, status, modules run.

### Risk Overview and Statistics

Five severity stat cards (Critical/High/Medium/Low/Info counts), a SVG donut chart of severity distribution, and a table of the most common vulnerability types.

### OWASP Top 10 Coverage

Maps every finding to the OWASP Top 10 2021 category. Shows which OWASP categories were found, count, and highest severity per category.

### Vulnerability Findings

Every non-informational finding displayed as a card with: title, severity badge, CVSS score, affected URL, vulnerable parameter, description, payload used, evidence snippet, remediation steps, and references.

### Informational Findings

Reconnaissance data (DNS records, tech stack, subdomain list) presented concisely.

### Remediation Priority Table

One-line summary of every finding sorted by severity — useful as a task list for the development team.

## 10.3 CVSS Scores Explained

CVSS (Common Vulnerability Scoring System) is a standardized way to score vulnerability severity from 0.0 to 10.0. Every finding in your framework has a CVSS score. Here is how to interpret them:

| Score Range | Severity Label | Meaning | Example |
|---|---|---|---|
| 9.0–10.0 | Critical | Immediate exploitation likely, severe impact | SQLi with DB access, default creds working |
| 7.0–8.9 | High | Exploitation likely, significant impact | XSS, missing HSTS, zone transfer |
| 4.0–6.9 | Medium | Exploitation possible with conditions | Username enumeration, no lockout, CSRF |
| 0.1–3.9 | Low | Limited impact or hard to exploit | Missing headers, info disclosure |
| 0.0 | Informational | No vulnerability — data only | DNS records, tech stack, subdomain list |

# 11. Security, Legal, and Ethical Considerations

## 11.1 The Most Important Rule

> ⚠ NEVER run this framework against a target you do not have explicit written authorization to test. This is not a moral guideline — it is the law.
>
> Unauthorized penetration testing is a criminal offense in most countries:
> • Bangladesh: ICT Act 2006, Section 54/56 — up to 7 years imprisonment
> • USA: Computer Fraud and Abuse Act (CFAA) — up to 10 years federal prison
> • UK: Computer Misuse Act 1990 — up to 10 years imprisonment
> • EU: Directive 2013/40/EU — criminal liability across all member states

## 11.2 Safe Practice Targets (Always Legal)

These are intentionally vulnerable applications designed for security training:

| Target | URL | How to Use |
|---|---|---|
| testphp.vulnweb.com | http://testphp.vulnweb.com | Live Acunetix test site — safe to scan anytime |
| testhtml5.vulnweb.com | http://testhtml5.vulnweb.com | HTML5 features test site |
| DVWA (Docker) | http://localhost (after docker run) | Most comprehensive local test app |
| OWASP Juice Shop | http://localhost:3000 (Docker) | Modern Node.js vulnerable app |
| WebGoat | http://localhost:8080 (Docker) | Java-based training app with lessons |
| HackTheBox | Various VPN IPs | Paid service — legal practice machines |
| TryHackMe | Various VPN IPs | Free/paid guided security labs |

```
# Run DVWA locally (safe target)
docker run -d -p 8080:80 vulnerables/web-dvwa
python cli.py scan http://localhost:8080 --type full

# Run Juice Shop locally
docker run -d -p 3000:3000 bkimminich/juice-shop
python cli.py scan http://localhost:3000 --type full
```

## 11.3 Built-in Safety Features

The framework has multiple layers of protection against accidental misuse:

- **Always-blocked list:** config.yaml has domains like google.com, amazon.com that can never be scanned regardless of user input

- **CLI legal notice:** Every scan requires explicit confirmation: "Do you confirm you have permission?" — cannot be bypassed
- **Scope enforcer:** scope.py prevents crawled URLs from leaving the target domain — discovered links to external sites are never followed
- **Rate limiting:** Default 5 req/s prevents the scanner from becoming a DoS tool
- **Request logging:** Every HTTP request is audit-logged with timestamp — creates accountability trail
- **No direct exploitation:** The framework detects vulnerabilities but does not exploit them (no reverse shells, no data exfiltration)

# 12. Configuration Reference

## 12.1 config.yaml — All Options

```yaml
# config.yaml — Base configuration
# Values here are defaults. Override any with .env variables.

app:
  name: "WAPT Framework"
  version: "0.1.0"

scan:
  default_rate_limit: 5    # requests per second
  default_timeout: 10      # seconds per request
  max_redirects: 5         # follow up to 5 redirects
  user_agents:             # rotated in polite/stealth mode
    - "Mozilla/5.0 (Windows NT 10.0; Win64; x64)..."

scope:
  always_blocked:          # NEVER scan these — hard blocked
    - "google.com"
    - "amazon.com"
    - "cloudflare.com"
  allowed_schemes:         # only http and https allowed
    - "http"
    - "https"

logging:
  level: "INFO"            # DEBUG | INFO | WARNING | ERROR
  rotation: "10 MB"        # rotate log file at 10MB
  retention: "7 days"      # keep logs for 7 days

reporting:
  severities:              # CVSS ranges for each severity level
    - name: "Critical"
      cvss_range: [9.0, 10.0]
    - name: "High"
      cvss_range: [7.0, 8.9]
```

## 12.2 .env — Environment Variables

| Variable | Default | Purpose |
|---|---|---|
| APP_ENV | development | production disables SQL logging, enables auth |
| SECRET_KEY | (change this) | API key for authentication in production |
| APP_HOST | 0.0.0.0 | IP address the server binds to |
| APP_PORT | 8000 | TCP port |
| DATABASE_URL | sqlite+aiosqlite:///./wapt.db | Database connection string |
| LOG_LEVEL | INFO | Logging verbosity |

| Variable | Default | Purpose |
|---|---|---|
| LOG_FILE | logs/wapt.log | Log file path |
| DEFAULT_RATE_LIMIT | 5 | Requests per second |
| DEFAULT_TIMEOUT | 10 | Request timeout in seconds |
| REPORT_OUTPUT_DIR | reports/output | Where to save generated reports |

# 13. Troubleshooting — Common Problems and Fixes

| Error | Cause | Fix |
|---|---|---|
| ModuleNotFoundError: No module named X | Package not installed | pip install -r requirements.txt |
| uvicorn api.main:app - could not import | Wrong module path | Use: uvicorn main:app --reload |
| sqlalchemy.exc.OperationalError | DB table does not exist | Run python cli.py server once to auto-create tables |
| httpx.ConnectError | Target unreachable | Check URL, try curl <url> first |
| nmap: command not found | Nmap not installed | apt install nmap (Linux) or brew install nmap (Mac) |
| WeasyPrint error on PDF | System libraries missing | apt install libpango-1.0-0 libharfbuzz0b |
| CORS error in browser | Frontend origin not whitelisted | Check CORS origins in main.py include http://localhost:5173 |
| Scan stays pending forever | API creates record but CLI runs scan | Run python cli.py scan <url> separately — API and CLI are separate |
| Port scan shows no results | Nmap needs root on Linux | sudo python cli.py scan <url> --type scan |
| crt.sh query failed | CT log API timeout | Normal — subdomain bruteforce still runs |
| Rate limit 429 error on target | Scanning too fast | Add --stealth polite or --rate-limit 2 |

# 14. What to Build Next — Extending the Framework

## 14.1 Immediate Improvements

### Connect API scan creation to actually running the scan

Currently, POST /api/scans creates a record but does not run the engine. Add a background task:

```
# In api/routes/scans.py — update create_scan():
@router.post("/", response_model=ScanResponse, status_code=201)
async def create_scan(
    data: ScanCreate,
    background_tasks: BackgroundTasks,
    db: AsyncSession = Depends(get_session)
):
    scan = Scan(...)
    db.add(scan)
    await db.flush()
    # Add this line to actually run the scan:
    background_tasks.add_task(run_scan_engine, scan.id, target.url, data)
    return scan
```

### Add WebSocket for real-time scan progress

Replace the 3-second polling in ScanDetail.jsx with a WebSocket connection that the engine writes progress to in real time.

## 14.2 New Vulnerability Modules to Add

| Module | What It Tests | Difficulty |
|---|---|---|
| XXE (XML External Entity) | Upload XML with entity references to read local files | Medium |
| SSRF (Server-Side Request Forgery) | Inject internal URLs to probe cloud metadata endpoints | Medium |
| Path Traversal | Inject ../../../etc/passwd in file parameters | Easy |
| Open Redirect | Inject external URLs in redirect parameters (example plugin exists) | Easy |
| HTTP Request Smuggling | Exploit discrepancy between proxy and server in header parsing | Hard |
| GraphQL Introspection | Query schema to enumerate all types, queries, mutations | Medium |
| JWT Brute Force | Crack weak HS256 secrets offline using wordlists | Medium |
| Prototype Pollution | Inject __proto__ in JSON parameters to corrupt object | Hard |

| Module | What It Tests | Difficulty |
|---|---|---|
| | properties | |

## 14.3 Infrastructure Improvements

- **PostgreSQL instead of SQLite:** For production use — set DATABASE_URL in .env to a PostgreSQL connection string
- **Redis for task queue:** Use Celery + Redis instead of FastAPI BackgroundTasks for reliable long-running scans
- **Authentication:** Add user login to the dashboard — JWT tokens, multiple user accounts, per-user scan history
- **Scheduling:** Add cron-style scheduled scans — check targets weekly automatically
- **Notifications:** Email or Slack alerts when a critical finding is discovered
- **CI/CD integration:** Run the scanner as part of your deployment pipeline — fail the build if critical findings exist

# 15. Quick Reference Card

## Everything in One Place

### Start the Stack

```
# Backend (from project root)
source venv/bin/activate
uvicorn main:app --reload --port 8000

# Frontend (new terminal)
cd ui && npm run dev

# Both via Docker
docker-compose up --build
```

### Run Scans

```
python cli.py scan http://testphp.vulnweb.com --type recon
python cli.py scan http://testphp.vulnweb.com --type full
python cli.py scan http://testphp.vulnweb.com --type full --stealth polite
python cli.py scan http://testphp.vulnweb.com --type full --rate-limit 2
```

### Generate Reports

```
python cli.py report <scan_id> --format all
python cli.py report <scan_id> --format pdf
python cli.py report <scan_id> --format markdown
```

### API Endpoints

```
GET    /api/health                     # Server status
GET    /api/docs                       # Swagger UI
POST   /api/targets/                   # Create target
GET    /api/targets/                   # List targets
GET    /api/targets/{id}               # Get target
DELETE /api/targets/{id}               # Delete target
POST   /api/scans/                     # Create scan
GET    /api/scans/                     # List scans
GET    /api/scans/{id}                 # Get scan
GET    /api/scans/{id}/findings        # Get findings
POST   /api/reports/generate           # Generate reports
GET    /api/reports/list               # List reports
GET    /api/reports/{scan_id}/summary # Quick summary
GET    /api/reports/download/{file}    # Download report
```

### Add a Plugin

```
# 1. Create plugins/my_check.py
# 2. Inherit WAPTPlugin
# 3. Set name, description, category
# 4. Implement async run() returning list of finding dicts
```

```
# 5. Use self.make_finding() to build findings
# 6. Done — auto-loads on next scan run
```

## Safe Test Targets

```
http://testphp.vulnweb.com          # Always available online
docker run -d -p 8080:80 vulnerables/web-dvwa    # Local DVWA
docker run -d -p 3000:3000 bkimminich/juice-shop  # Local Juice Shop
```